



Algoritmo e Estrutura de Dados I

Ana Vitória S. da Luz
Gabriel Oliveira Ponde
Lara Côrtes Japiassu
Lucas Garcia de Lacerda
Nicolas Alves de Jesus



Sumário

1. Sumário Executivo
- 2 .O problema que o algoritmo resolve
- 3 Como o algoritmo funciona (explicação e fluxograma)
4. Pseudocódigo do algoritmo
5. Código-fonte em C
6. Complexidade do algoritmo (Notação Big-O)
7. Aplicação real na Ciência da Computação
8. Resultados obtidos
9. Limitações da solução
12. Anexos (Fluxogramas)

Sumário Executivo

1. O problema que o algoritmo resolve

O algoritmo Selection Sort resolve o problema de organizar pequenas amostras de dados de engajamento (como curtidas e interações) em redes sociais. Isso permite identificar quais publicações tiveram melhor desempenho, detectar valores discrepantes (outliers) e selecionar os conteúdos mais representativos para análise ou treinamento inicial de modelos de recomendação.

2. Funcionamento do código

Este código implementa o algoritmo de ordenação Selection Sort, a fim de organizar um conjunto de dados de engajamento em ordem crescente. Além da ordenação, o programa realiza uma análise estatística simples para identificar valores discrepantes, calculando o elemento que apresenta maior variação absoluta em relação à média do vetor em questão. O resultado é uma ferramenta que não apenas classifica a informação, mas destaca anomalias relevantes sobre os dados.

3. Pseudocódigo

INÍCIO FUNÇÃO OrdenarEngajamento (recebe: vetor $v[]$, tamanho n)

 PARA i DE 0 ATÉ $n-2$ FAÇA

 menor_indice $\leftarrow i$

 PARA j DE $i+1$ ATÉ $n-1$ FAÇA

 SE $v[j] < v[\text{menor_indice}]$ ENTÃO

 menor_indice $\leftarrow j$ // Atualiza o índice do menor encontrado

 FIM SE

 FIM PARA

 variavel_auxiliar $\leftarrow v[i]$

$v[i] \leftarrow v[\text{menor_indice}]$

$v[\text{menor_indice}] \leftarrow \text{variavel_auxiliar}$

 FIM PARA

FIM FUNÇÃO

INÍCIO FUNÇÃO AcharDiscrepante (recebe: vetor $v[]$, tamanho n) RETORNA Inteiro

 soma $\leftarrow 0$

 PARA i DE 0 ATÉ $n-1$ FAÇA

 soma $\leftarrow \text{soma} + v[i]$

 FIM PARA

 media $\leftarrow \text{soma} / n$

discrepante <- v[0]

maior_distancia <- ValorAbsoluto(v[0] - media)

PARA i DE 1 ATÉ n-1 FAÇA

 distancia_atual <- ValorAbsoluto(v[i] - media)

 SE distancia_atual > maior_distancia ENTÃO

 maior_distancia <- distancia_atual

 discrepante <- v[i]

 FIM SE

FIM PARA

RETORNE discrepante

FIM FUNÇÃO

INÍCIO FUNÇÃO Mostrar (recebe: vetor v[], tamanho n)

 PARA i DE 0 ATÉ n-1 FAÇA

 ESCREVA v[i] + " "

 FIM PARA

 PULAR_LINHA

FIM FUNÇÃO

INÍCIO ALGORITMO Principal

vetor engajamento <- [120, 88, 540, 95, 110, 600, 102]

n <- TamanhoDe(engajamento)

ESCREVA "1. Levantamento de engajamento: "

CHAMAR Mostrar(engajamento, n)

CHAMAR OrdenarEngajamento(engajamento, n)

ESCREVA "2. Dados organizados: "

CHAMAR Mostrar(engajamento, n)

valor_discrepante <- CHAMAR AcharDiscrepante(engajamento, n)

ESCREVA "Discrepância detectada: ", valor_discrepante

FIM ALGORITMO

4. Código em C

```
#include <stdio.h>
#include <math.h>

void ordenarEngajamento(int v[], int n)
{
    for (int i = 0; i < n - 1; i++){
        int menor = i;

        for (int j = i + 1; j < n; j++){
            if (v[j] < v[menor]){
                menor = j;
            }
        }

        int aux = v[i];
        v[i] = v[menor];
        v[menor] = aux;
    }
}

int acharDiscrepante(int v[], int n)
{
    float soma = 0;
    for (int i = 0; i < n; i++) {
        soma += v[i];
    }

    float media = soma / n;
    int discrepante = v[0];
    float maiorDist = fabs(v[0] - media);

    for (int i = 1; i < n; i++){
        float dist = fabs(v[i] - media);
        if (dist > maiorDist){
            maiorDist = dist;
            discrepante = v[i];
        }
    }
    return discrepante;
}

void mostrar(int v[], int n)
```

```

{
    for (int i = 0; i < n; i++){
        printf("%d", v[i]);
    }
    printf("\n");
}

int main()
{
    int engajamento[] = {120, 88, 540, 95, 110, 600, 102};
    int n = sizeof(engajamento) / sizeof(engajamento[0]);

    printf("\n1. Levantamento de engajamento: ");
    mostrar(engajamento, n);
    ordenarEngajamento(engajamento, n);

    printf("\n2. Dados organizados: ");
    mostrar(engajamento, n);

    int discrepante = acharDiscrepante(engajamento, n);
    printf("\nDiscrepância detectada: %d\n", discrepante);

    return 0;
}

```

5. Complexidade

A complexidade do Selection Sort é $O(n^2)$ devido às duas estruturas de repetição aninhadas. O primeiro laço percorre o vetor desde a primeira até a penúltima posição. O segundo, por sua vez, percorre o vetor a partir da segunda posição e vai até a última. Isso ocorre pois o algoritmo precisa comparar um elemento com todos os outros, por isso, o laço interno inicia sempre uma posição a mais que a atual do laço externo, isto é, caso o laço externo esteja com $i=0$, o laço interno começará com $j=1$, e assim por diante. Assim, quando o índice externo está na primeira posição, o laço interno realiza aproximadamente $n-1$ comparações. Na segunda posição, realiza $n-2$ comparações, e assim sucessivamente, até que reste apenas uma comparação na última passagem. A quantidade total de comparações forma uma soma aritmética representada por:

$$(n - 1) + (n - 2) + \dots + 1 = \frac{n(n-1)}{2}$$

Portanto, o termo dominante dessa equação é n^2 , já que para valores grandes de n , os termos inferiores podem ser desprezados. Logo, o tempo de execução cresce quadraticamente em relação ao tamanho da entrada.

6. Aplicação real na Ciência da Computação

A aplicação prática do algoritmo na Ciência da Computação está no pré-processamento de dados de engajamento, fundamental para análise, detecção de anomalias e preparação de dados para sistemas de recomendação e machine learning. A combinação de ordenação e identificação de discrepantes permite seleccionar conteúdos relevantes, identificar picos de engajamento e reduzir ruído nos dados. Trata-se de uma etapa frequentemente usada em sistemas de marketing digital, dashboards de BI e modelos iniciais de recomendação, além de ser um exemplo didático importante para compreensão de algoritmos clássicos.

7. Resultados

A implementação do algoritmo apresenta três resultados principais:

1. Ordenação correta dos dados de engajamento

O Selection sort organiza o vetor de forma crescente, permitindo visualizar facilmente:

1. quais publicações tiveram menor ou maior engajamento,
2. a distribuição geral dos dados,
3. possíveis padrões ou tendências.

Isso torna a análise inicial mais intuitiva e facilita etapas posteriores, como seleção de conteúdo relevante.

2. Identificação de valores discrepantes (outliers)

A função de detecção de discrepante calcula a média e identifica o item que se afasta mais dela. Esse resultado é útil para:

1. detectar os picos de engajamento,
2. identificar posts que performaram muito acima ou muito abaixo do normal,
3. destacar conteúdos que merecem investigação.

3. Suporte ao pré-processamento de dados

A combinação de ordenação + detecção de discrepantes gera um conjunto de dados mais limpo e organizado, pronto para:

1. análises estatísticas básicas,
2. visualizações,
3. ou para servir como entrada inicial em modelos de machine learning.

7. Limitações

Apesar dos resultados positivos, a solução apresenta limitações importantes:

1. Complexidade alta ($O(n^2)$)

O Selection Sort não é eficiente para grandes volumes de dados.

Seu custo quadrático faz com que:

1. o tempo de execução cresça rapidamente,
2. o algoritmo seja inviável para bases reais de redes sociais, que possuem milhões de pontos.

É adequado apenas para pequenas amostras, como foi demonstrado.

2. Detecção de discrepantes é simplificada

O método utilizado (distância absoluta da média):

1. não considera desvio padrão,
2. não trata casos com múltiplos outliers,
3. não identifica se o valor discrepante é realmente um erro ou apenas um pico legítimo.

3. Não trata dados ruidosos ou inconsistentes

O algoritmo assume que:

1. todos os valores são válidos,
2. não há dados faltantes,
3. não existem erros de coleta.

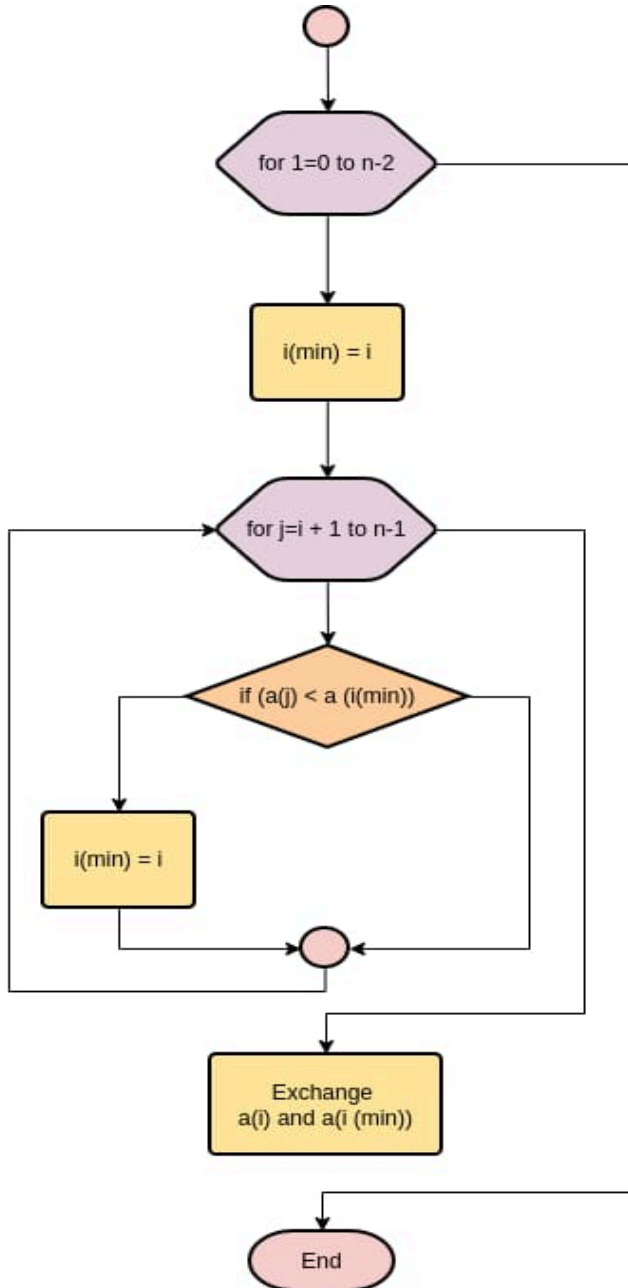
Sistemas reais precisam lidar com:

1. valores nulos,
2. duplicados,
3. dados inconsistentes,
4. escala diferente entre métricas.

O algoritmo cumpre seu objetivo de ordenar pequenas amostras de engajamento e identificar valores discrepantes, oferecendo uma base inicial para análise de dados. Entretanto, sua complexidade elevada, a simplicidade do método de detecção de outliers e a ausência de contexto e robustez limitam sua aplicação em cenários reais de grande escala. Ainda assim, ele funciona bem como ferramenta didática e como protótipo de pré-processamento de dados.

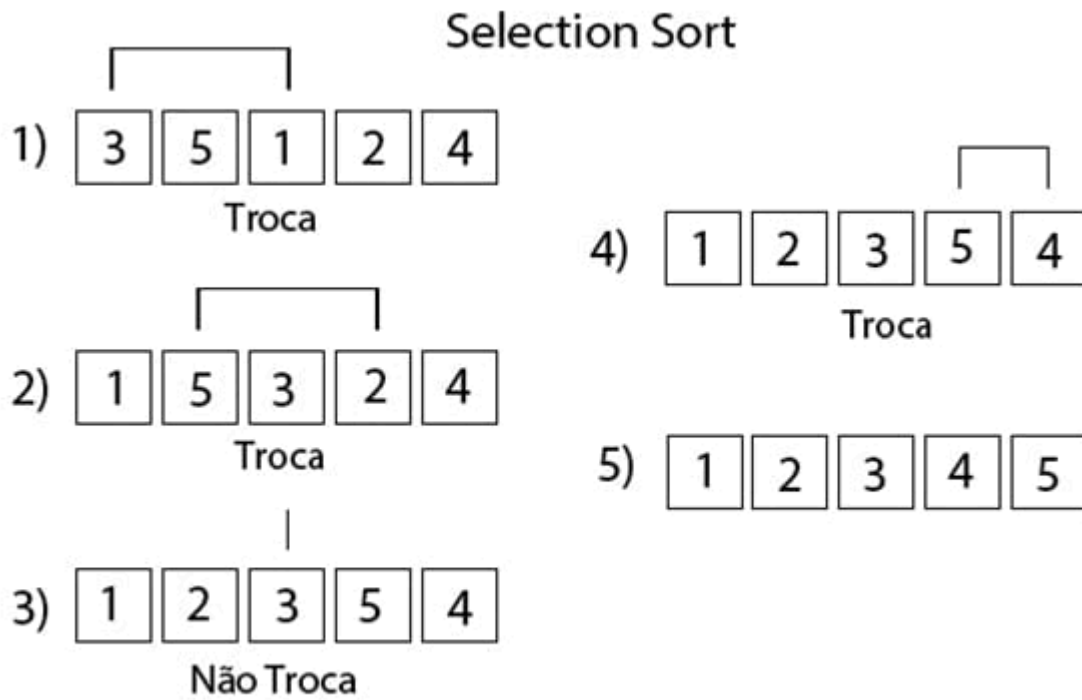
Fluxograma:

O fluxograma descreve o processo de encontrar o menor elemento de uma lista e colocá-lo na posição correta, repetindo isso até ordenar todo o vetor.



O outro fluxograma de exemplo abaixo mostra que o Selection Sort:

1. Escolhe o menor elemento da parte não ordenada
2. Coloca esse elemento na frente
3. Repete isso para cada posição
4. Faz no máximo uma troca por iteração



Por fim, abaixo se encontra o gráfico que ilustra a complexidade presente neste algoritmo, o que destaca o escopo limitado no qual ele pode ser implementado.

